CS 663 Assignment 3

Team Members: *Siddhesh Phadke (24b4502), Kushagra Kushwaha (24b2191)*

All the outputs are present in the submission/output folder.

Code for each question is in its folder (Q1, Q2, Q3, Q4); every script runs from the Assignment 3 folder and writes its figures to output/.

Question 1:

Mean shift on joint colour + position features gives 2 segments for bison and leopard, and 4 for ghats.

(a) Mean-shift segmentation. Each pixel is the feature vector [R, G, B, s·X, s·Y], with X and Y scaled to 0–255 and s the spatial weight. The feature density is modelled with a Gaussian kernel (bandwidth h) on a random subset of pixels. Each sample is shifted to its density mode with the sample set held fixed. Modes closer than 0.6h are merged, and at most max_segments are kept.

(b) Label image and boundaries. Every pixel takes the segment of its nearest density sample, giving labels 1 to N. Boundaries are pixels where the label changes, dilated to 2 pixels and drawn in black on the input.

Image	Spatial weight s	Bandwidth h	Colour pre-smoothing σ	Samples	Max iterations / tolerance	Segments
bison.png	0.20	30	none	1500	20 / 0.5	2
ghats.png	0.50	30	none	2000	15 / 0.5	4
leopard.png	0.40	18	12 px	2000	300 / 0.1	2

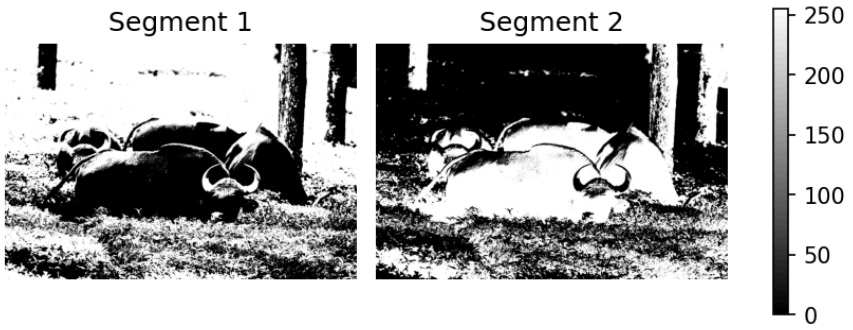
For the leopard, the spots split the coat into orange and black pixels, so it never forms its own colour mode. Smoothing the colours (σ = 12 px) before building the features averages the spots into one coat colour. Mean shift is also run until it converges, so the grass and leopard modes separate.

(c) Results. For each image: the input beside its segmentation map, the binary mask of every segment, and the boundaries drawn on the input.

bison.png



bison.png: Binary segment masks



bison.png: Segment boundaries

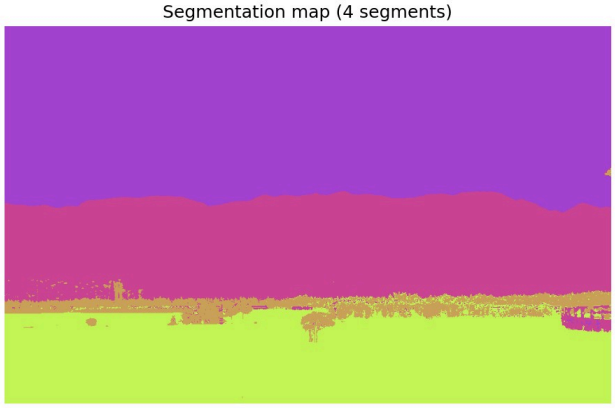


ghats.png

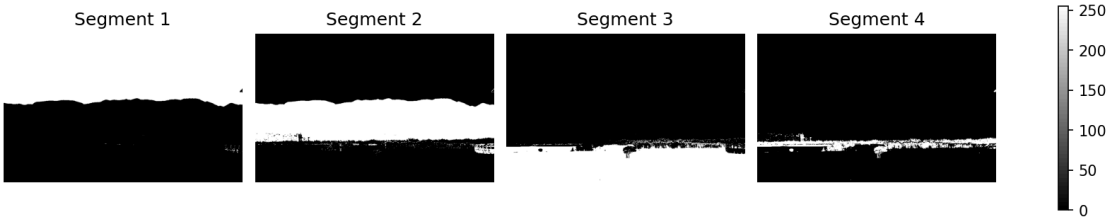
ghats.png: Original



Segmentation map (4 segments)



ghats.png: Binary segment masks



ghats.png: Segment boundaries



leopard.png

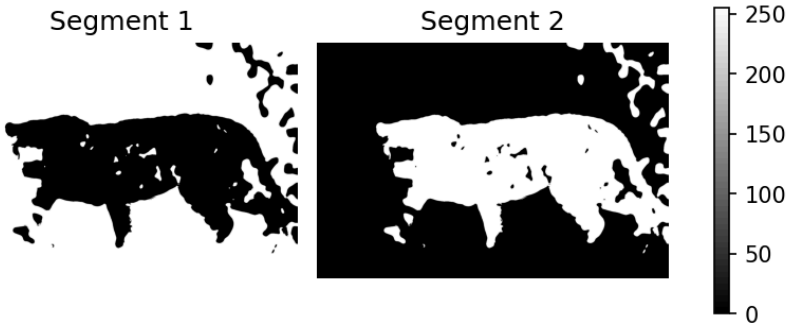
leopard.png: Original



Segmentation map (2 segments)



leopard.png: Binary segment masks



leopard.png: Segment boundaries



Question 2:

Both filters visibly reduce the noise while keeping major edges; Perona–Malik removes more noise, the bilateral filter keeps more fine texture.

All four given images are already noisy and no clean version exists, so the original image is the noisy input. Intensities are floats in [0, 1].

(a) Perona–Malik anisotropic diffusion. Each iteration applies the discretisation from class, with differences to the 4 neighbours and zero flux across the image border:

$$I^{new}(x,y) = I(x,y) + \lambda\,[\,c_N\nabla I_N + c_S\nabla I_S + c_E\nabla I_E + c_W\nabla I_W\,], \qquad c(\nabla I) = \frac{1}{1+(\|\nabla I\|/K)^2}$$

The time step is λ = 0.2, under the 0.25 stability limit. For colour images, $\|\nabla I\|$ is the norm of the RGB difference, so all three channels share one coefficient and edges stay aligned. The inverse-quadratic c(·) worked better than $\exp(-(\|\nabla I\|/K)^2)$, which left isolated noisy specks.

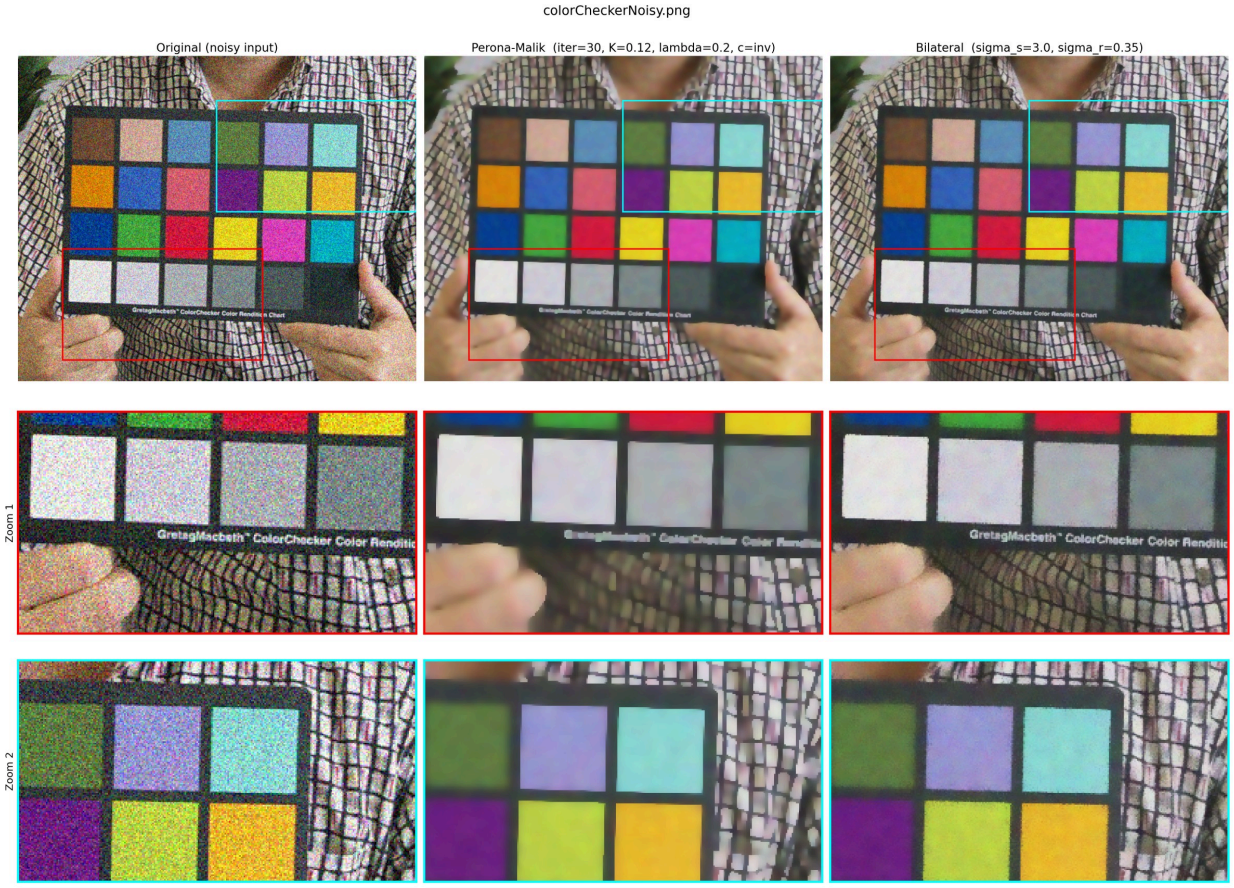
(b) Bilateral filter. Each pixel becomes a normalised average weighted by both spatial distance and intensity difference:

$$BF[I]_p = \frac{1}{W_p}\sum_{q\in S} G_{\sigma_s}(\|p-q\|)\,G_{\sigma_r}(\|I_p-I_q\|)\,I_q, \qquad W_p = \sum_{q\in S} G_{\sigma_s}(\|p-q\|)\,G_{\sigma_r}(\|I_p-I_q\|)$$

The window S is a disc of radius $3\sigma_s$, and $\|I_p - I_q\|$ is the Euclidean distance between RGB colours.

Image	P–M iterations	P–M K	P–M c(·)	Bilateral σ_s (px)	Bilateral σ_r
colorCheckerNoisy.png	30	0.12	inverse quadratic	3	0.35
rajaRaviVarma.png	20	0.05	inverse quadratic	3	0.18
sagarmaathaa.png	20	0.035	inverse quadratic	3	0.12
racing.png	30	0.07	inverse quadratic	3	0.25

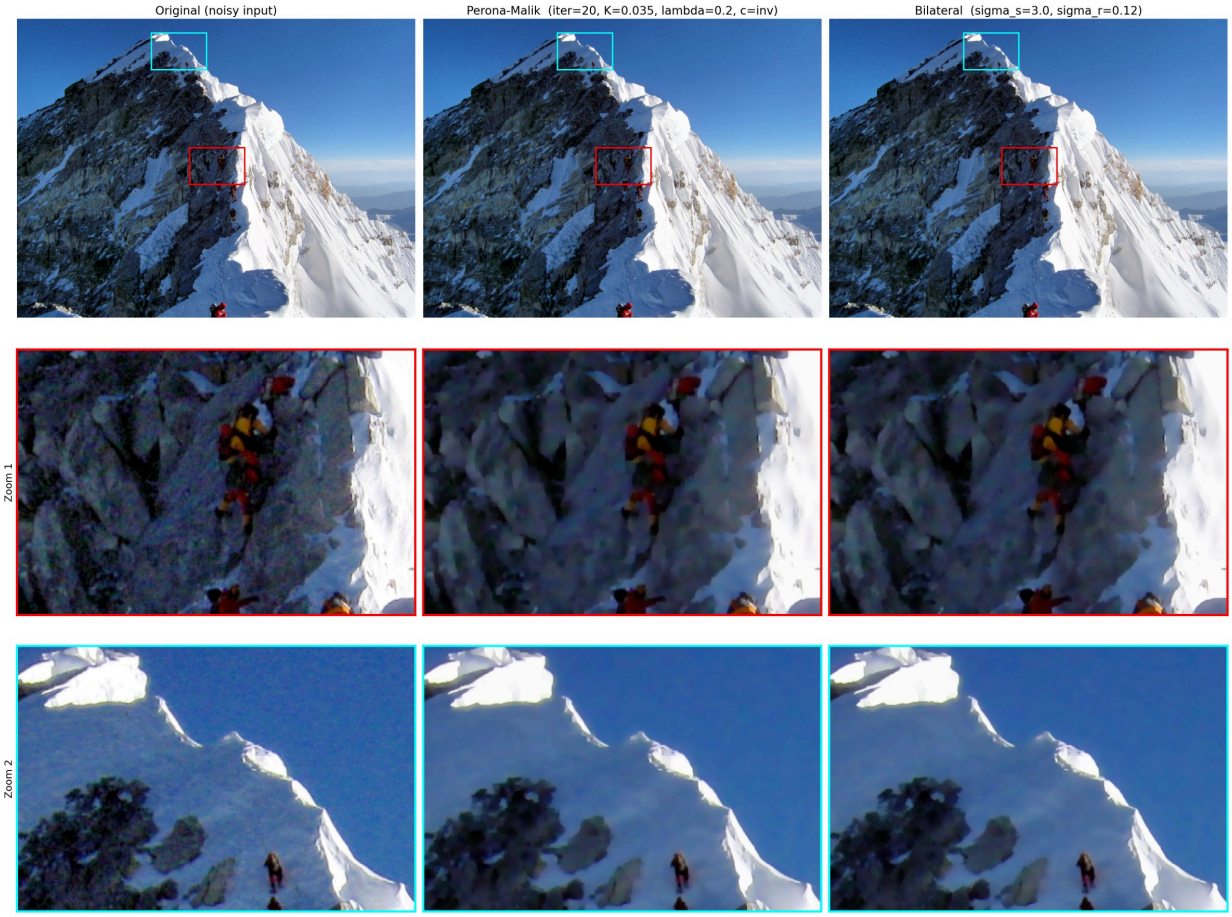
(c) Results. Each figure shows the original (noisy) image, the Perona–Malik result and the bilateral result, with two zoomed regions per image. The red and cyan boxes on the full images mark the zoomed regions.

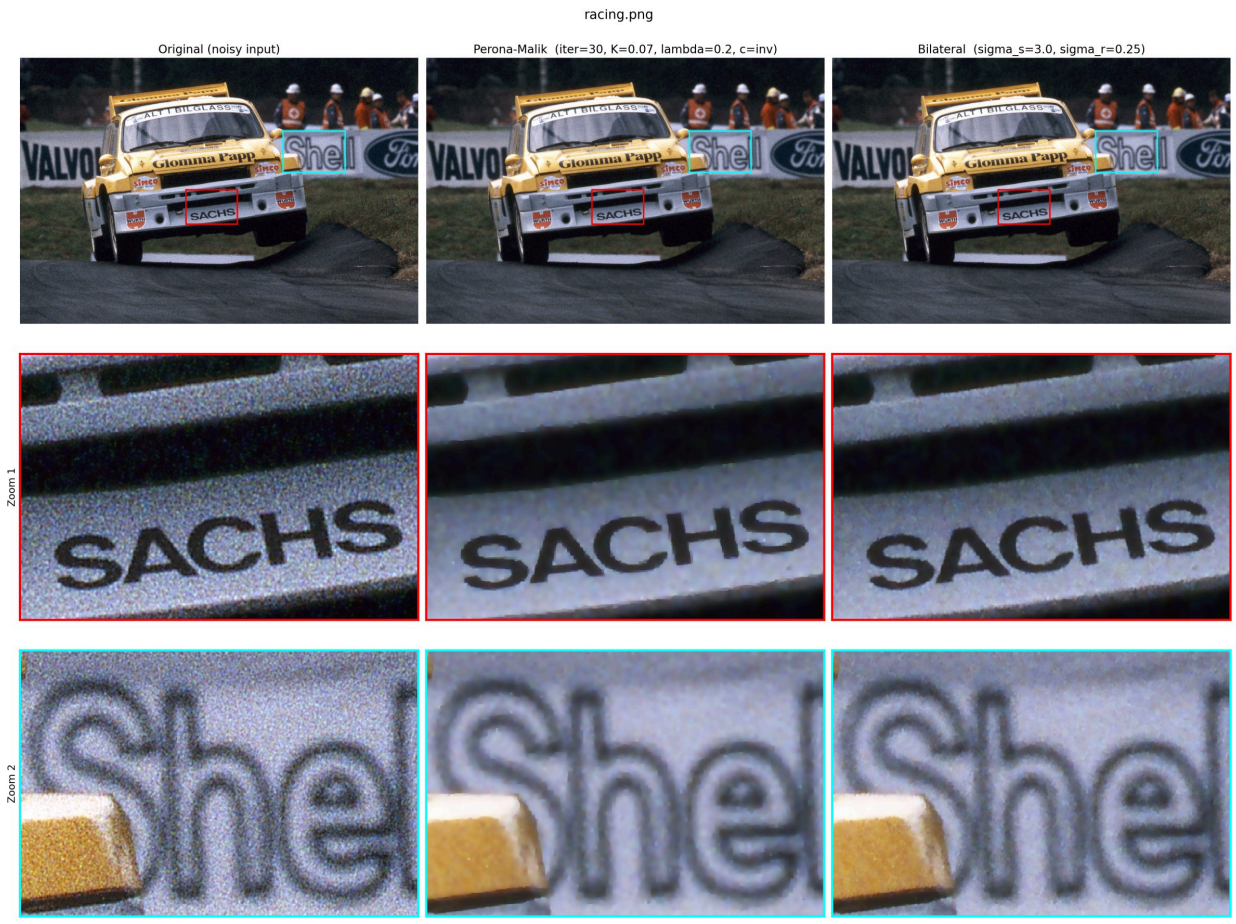


rajaRaviVarma.png



sagarmaathaa.png





Perona–Malik with these settings looks slightly flat in textured areas such as the rock face and the stamp's background. The bilateral filter keeps that texture but leaves some grain behind.

Question 3:

Patch-based texture synthesis grows all three samples to 256 × 256 in about 35 s each, and the same matching removes all text from lohagad.png in about 40 s.

(a) Texture synthesis. The 80 × 80 sample is placed at the centre of the 256 × 256 output, and the texture grows outwards:

1. Pick the unknown pixel whose 13 × 13 neighbourhood has the most known pixels (random among ties).
2. Compare that partly known neighbourhood with every 13 × 13 patch of the sample, using a Gaussian-weighted sum of squared differences over the known pixels.
3. Choose one patch at random among those within 1.1 × the best distance.
4. Copy its values into all unknown pixels of the neighbourhood, then repeat until the output is full.

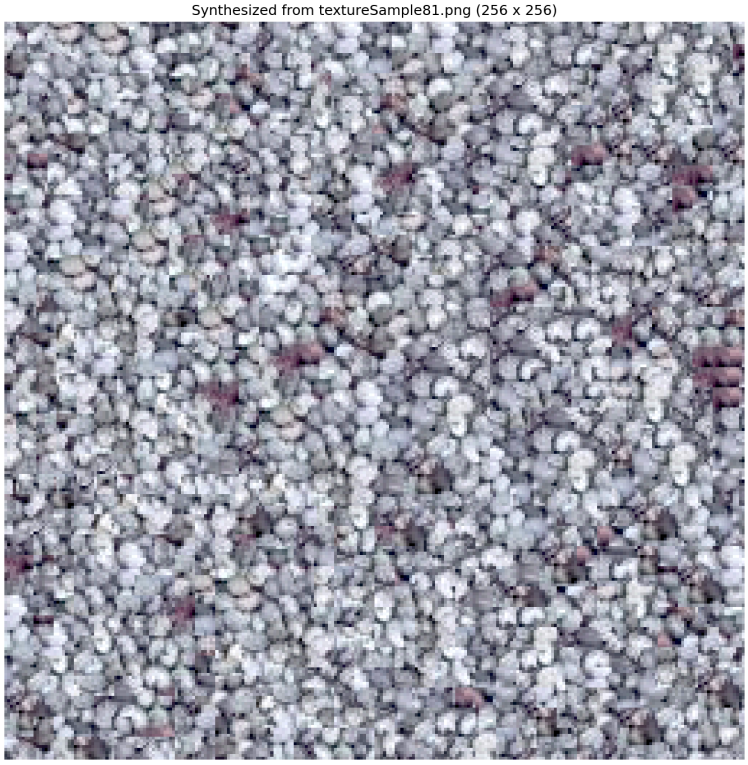
Filling the most complete neighbourhood first is what keeps long structures intact: a random fill order broke the stripes of textureSample207 into blotches.

(b) Results. Each sample is shown at the same pixel scale as its synthesised texture.

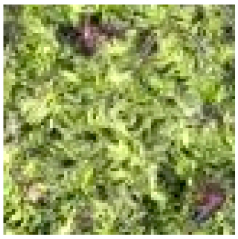
Sample
(80 x 80)



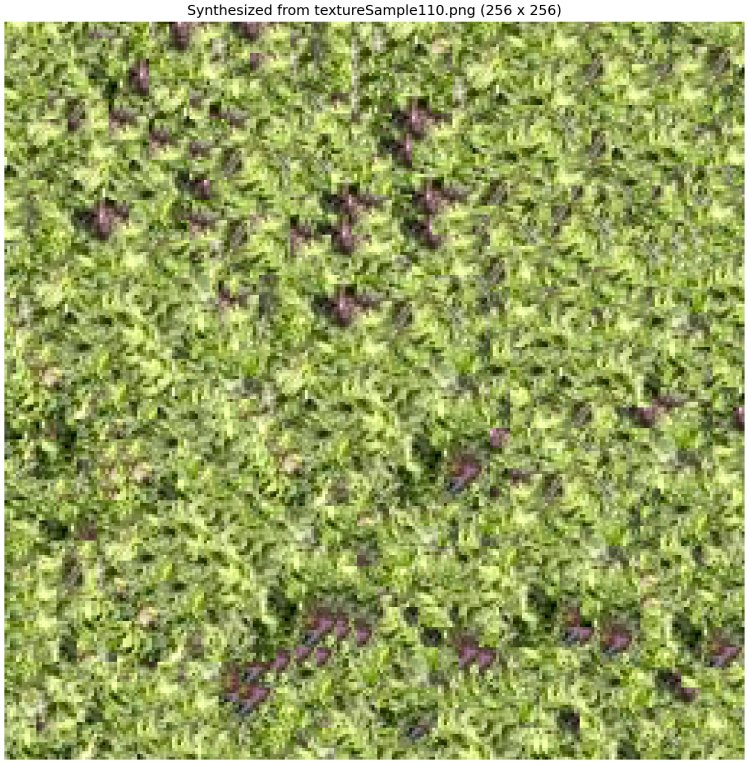
Synthesized from textureSample81.png (256 x 256)

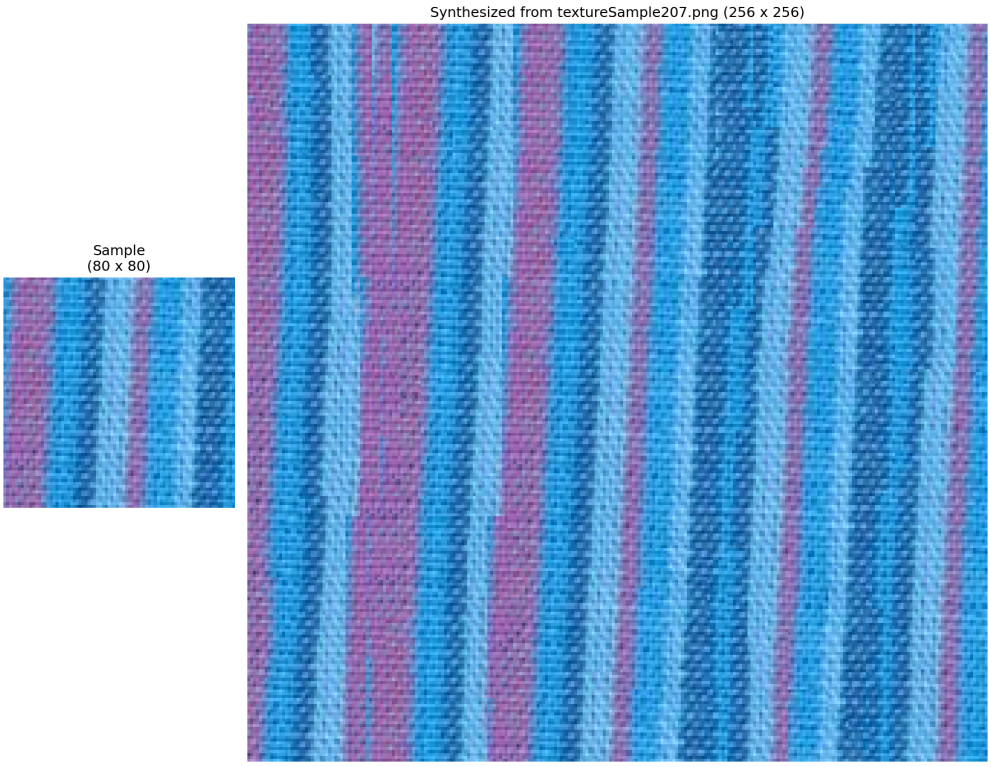


Sample
(80 x 80)



Synthesized from textureSample110.png (256 x 256)





(c) Inpainting. The hole is filled with the same steps, using the known part of the image as the sample. Candidates are every 7 × 7 patch that lies fully outside the hole within 40 px of the neighbourhood being filled. That way each part of the hole is filled from the texture around it (rock, trees or cliff). Pixels outside the hole are never changed.

(d) Text removal on lohagad.png. The hole is found automatically and kept tight around the labels:

1. Seeds: near-white, unsaturated pixels (letters, pin rims, pin icons) and saturated purple pixels (pin bodies).
2. Seeds are grouped by a 15 px dilation; groups with fewer than 150 seed pixels are dropped, which removes isolated bright specks on the rock.
3. The kept seeds are grown by 4 px to cover the dark outline of the letters.
4. Unmasked islands smaller than 40 px inside letters (such as the gaps in a, d, E) are added to the hole.

The hole covers 2.3% of the image. The two location pins are removed along with the text, since they are part of the same map overlay.



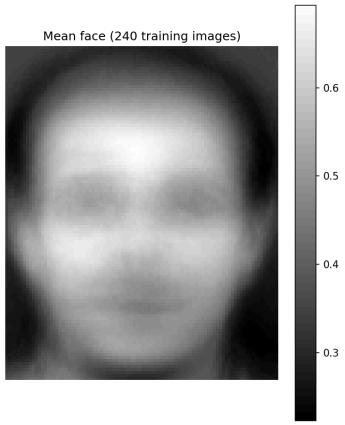
A 2–3 px sliver of cut-off letters along the bottom edge is not removed. Where the second pin stood, the filled-in cliff edge is slightly jagged.

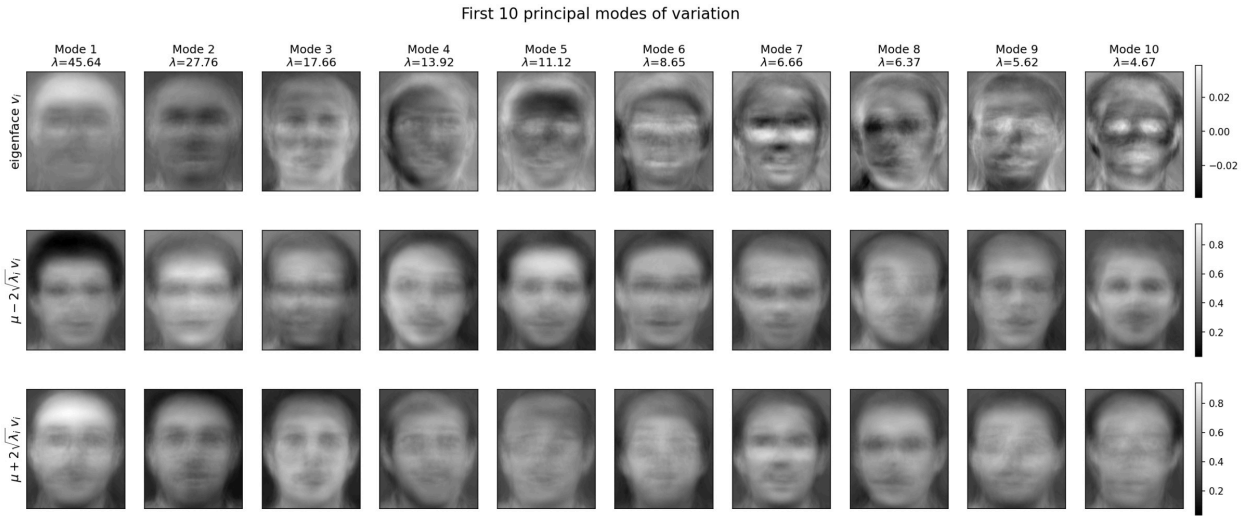
Question 4:

The tuned face-recognition system makes 141 of 160 test decisions correctly (88.1%), using 26 eigenfaces.

The dataset contains planted non-faces: s38 holds upside-down faces, s39 faces buried in salt-and-pepper noise, and s40 intensity-negated faces (its README says 37 distinct subjects). These count as “not a face”. The training set is images 1–8 of subjects 1–30 (240 images); the test set is the other 160 images.

(a) Mean and principal modes. Images are flattened to 10304-vectors with intensities in [0, 1]. The covariance matrix is 10304 × 10304, so its eigenvectors are found from the 240 × 240 matrix X^TX / N: if w is an eigenvector of it, Xw is an eigenvector of the covariance with the same eigenvalue. Each mode is shown as the eigenface v_i and as $\mu \pm 2\sqrt{\lambda_i}\, v_i$.





(b) Distance from face space. The score is the part of the mean-subtracted image that the k eigenfaces cannot represent:

$$\varepsilon(x) = \| (x - \mu) - U_k U_k^T (x - \mu) \|$$

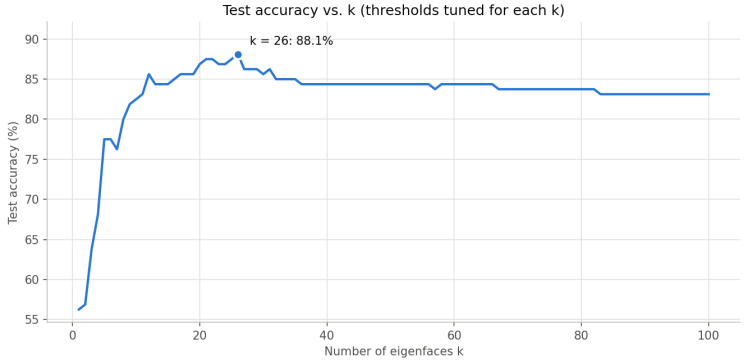
M1 holds this score for all 40 × 10 images.

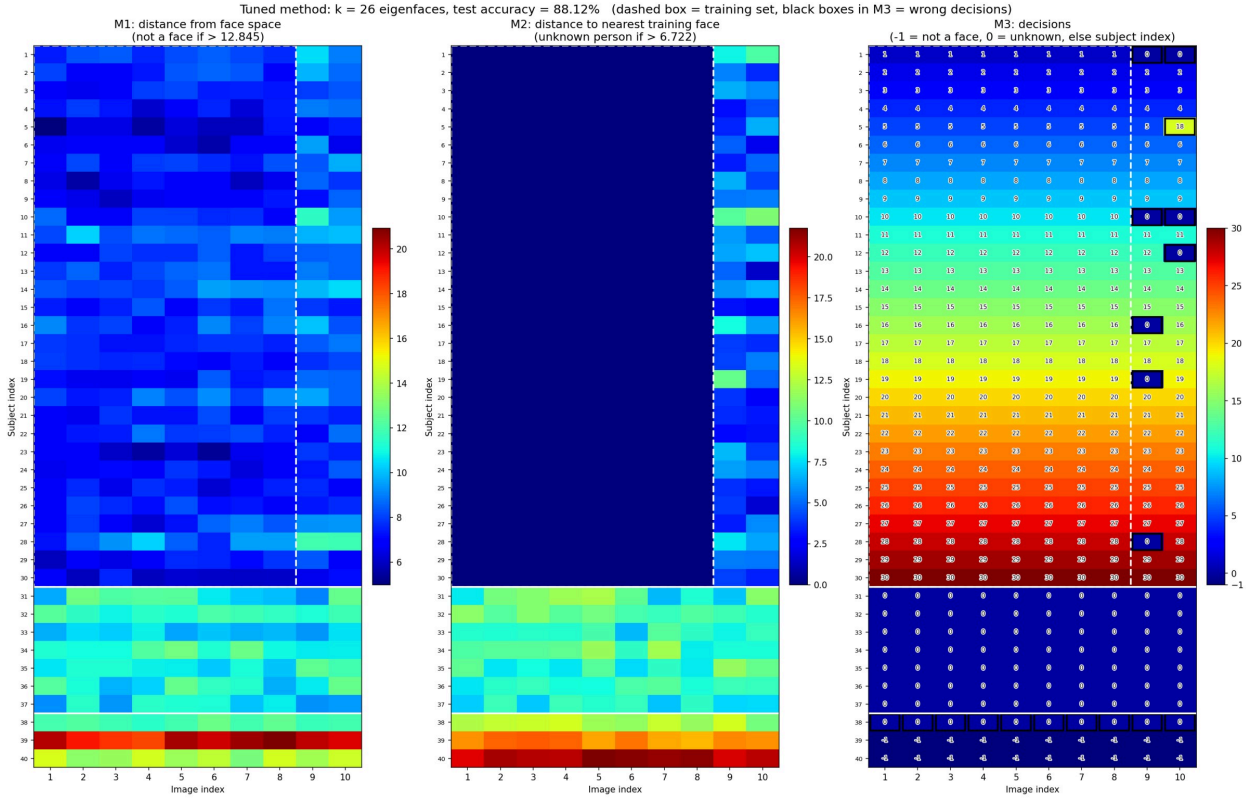
(c) Recognition. The face-space projection of an image is its weight vector $U_k^T(x - \mu)$. The classifier returns the identity of the nearest training projection, and M2 holds the distance to it. Training images get 0, since their nearest projection is their own.

(d) Face recognition on the test set. The decision is −1 if M1 > θ_1 (not a face), else 0 if M2 > θ_2 (a face not in the training set), else the index of the nearest training subject. The number of eigenfaces k (1 to 100) and both thresholds were searched exhaustively for the best test accuracy: k = 26, θ_1 = 12.845, θ_2 = 6.722.

Test group	**Images**	**Correct**
Known faces (s1–30, images 9–10)	60	51
Unknown faces (s31–37)	70	70
Non-faces (s38–40)	30	20
Total	**160**	**141 (88.1%)**

Of the 9 wrong known faces, 3 are matched to the wrong person and 6 are rejected as unknown. On all 400 images (training + test), 95.25% of decisions are correct.





The salt-and-pepper (s39) and negated (s40) images lie far from face space and are rejected. The upside-down faces (s38) score 11.4–12.7, which overlaps the real unknown faces, so no threshold on distance from face space separates them.